PU Assist

Multimodal RAG chatbot for Presidency University, Bengaluru – project notes

- **Inside:** notebook cell guide · errors and fixes · open gaps · parameters to tune · tech stack · other models
- **Platform:** Data Lab JupyterHub · NVIDIA H200 NVL (143 GB GPU memory) · Docker Swarm
- **Code and data:** github.com/shreshav2404-code/itw_pu_chatbot-with-RAG-implemenattion
- **Date:** 30 October 2026

At a glance

- Answers student questions from the university's own documents, with [1] [2] citations
- Input: typed text, photos (notices, timetables, assignments) and voice notes (English, Hindi, Kannada, …)
- Knowledge base: 166,180 cleaned chunks (website, 2,522 documents, repository, library, map, tour, media)
- RAG instead of fine-tuning: no training cost, update the data any time
- Measured on the H200: model load 52 s / 62.1 GB · index build 7.6 min (~350 chunks/s) · search ~0.3 s
- Live: public Streamlit page through a Cloudflare link (notebook 03, step 9), used for a class test

1. How the pieces fit together

File	Role	When to run
01_model_setup.ipynb	Installs libraries, downloads the vision-language model, tests text + image	Once per hub server start (steps 1–4)
02_rag_chatbot.ipynb	Builds/loads the search index; the full chatbot; tests; chat box	After 01; builds the index the first time
03_backend_api.ipynb	Same pipeline served as an API (FastAPI) + public links (Cloudflare / Gradio)	After 02, with 02's kernel shut down
frontend/	Streamlit chat page (Docker image for Portainer, or started by 03 step 9)	For the student-facing page
pu_corpus.jsonl.gz	The knowledge base: 166,180 chunks with source, type, topics	Uploaded next to the notebooks
campus_map.png	Campus map given to the model for "where is …" questions	Uploaded next to the notebooks

Run order on the hub

- Upload the files from D:\pu_bot\upload into one hub folder (pu_bot)
- 01: run steps 1–2 → **Kernel → Restart** → run steps 3–4 → shut down 01's kernel
- 02: **Restart Kernel and Run All** (first run builds the index, ~8 min)
- Shut down 02's kernel (two copies of the 62 GB model do not fit) → 03: **Run All**
- Public page: in 03 set `MAKE_PUBLIC_LINK = True` (step 9) or `MAKE_GRADIO_LINK = True` (step 10) and run that cell
- Stopping the hub server wipes files, model, index and libraries → repeat from the upload

How one question is answered

- **Listen:** a voice note → Whisper large-v3-turbo → text (keeps the spoken language)
- **Understand:** Qwen3-VL rewrites the question in the documents' wording (+ chat history); reads an uploaded image
- **Retrieve:** Qwen3-Embedding vectors → FAISS top 50 per query → bge-reranker picks the best 6 passages
- **Answer:** Qwen3-VL writes the answer from those passages (+ image, + campus map for directions) with citations

2. Notebook 01 – setup and model check (19 cells, 9 code)

Step	What the cell does	Notes
1. Check the machine	Prints GPU name and memory (nvidia-smi), RAM, disk, versions of torch, transformers, faiss, …	Seconds
2. Install libraries	Upgrades transformers, accelerate, sentence-transformers, gradio; pins `torch==2.9.1`; installs `torchvision==0.24.1` (CUDA 12.8)	3–5 min · pip conflict warnings are harmless · **restart the kernel after**
3. Versions + choose model	Sets `MODEL_ID` / `MODEL_DIR`; prints versions; checks the model class is supported by `AutoModelForImageTextToText`	Change `MODEL_ID` here to use another model
4. Download the model	`snapshot_download` to ~/hf_models/<model name>	~62 GB, 5–20 min · resumable
5. Load onto the GPU	Loads model (`dtype="auto"` = bf16, `cuda:0`, SDPA attention) + processor	~1 min · 62.1 GB measured
6. Helper functions	`load_image()` shrinks photos to 1600 px; `ask_model()` = chat template → generate (greedy) → decode, prints tok/s	Speed shows here
7. Test: text question	Asks a general question to confirm generation works	~seconds
8. Test: image question	Reads an uploaded image (default admissionoffice.jpg) and describes it	Upload an image first
9. GPU memory	Prints allocated / peak / total GPU memory	Shut the kernel down afterwards

3. Notebook 02 – the RAG chatbot (41 cells, 21 code)

A. Search index

Step	What the cell does	Notes
1. Settings	Paths; `MODEL_ID`, `EMBED_MODEL`, `RERANK_MODEL`, `ASR_MODEL`; `CANDIDATES=50`, `TOP_K=6`, `MAX_PER_SOURCE=3`, `MAX_UNOFFICIAL=3`, `CONTEXT_CHARS=300`, `REBUILD_INDEX`, device	All tuning knobs in one place
2. Load the corpus	Reads pu_corpus.jsonl.gz (166,180 chunks); prints counts per source class	Seconds
2b. Chunk context	Builds `passages` = title + last 300 characters of the previous chunk + text (also links PDF pages)	Stops headings being lost at chunk boundaries
3. Search models	Qwen3-Embedding-0.6B (bf16, max 512 tokens) + bge-reranker-v2-m3; query instruction prompt; `embed()` normalises vectors	~30 s
4. FAISS index	Loads rag_index/ or builds it: embeds all passages in blocks of 10,000 → `IndexFlatIP` (exact cosine) → saves index + info.json	First time 7.6 min · auto-rebuild when corpus / embedder / passage format change
5a. search()	Source labels/links; embeds question + rewrites → FAISS top 50 each → union → rerank → caps per document and unofficial sources; prints sample hits	~0.3 s per question on the H200
5b. Retrieval test	13 test questions: is the right source in the top 6? prints rank + score X/13	Plain search (no rewriting)

B. Chatbot

Step	What the cell does	Notes
6. Load the VLM	Downloads `MODEL_ID` if missing, loads it with `AutoModelForImageTextToText`	~1 min · 62 GB
7. Helpers	`load_image()` and `ask_model()` (same as notebook 01)	
7b. Voice (2 cells)	`%pip install av` (no restart needed); Whisper large-v3-turbo pipeline (fp16, 30 s chunks); `load_audio()` decodes any format to 16 kHz mono; `transcribe()`	1–2 min first time
8. RAG pipeline	`SYSTEM_PROMPT` (9 rules), `REWRITE_PROMPT`/`rewrite_query()`, `read_image()`, `build_context()`, `format_sources()`, `answer_question()` (listen → understand → search → answer → citations), `chat()`	Edit prompts here

C. Try it

Step	What the cell does	Notes
9. Retrieval check + rewriting	Same 13 questions, each rewritten by Qwen3-VL first; prints rewrites + score X/13	Compare with step 5b
10. Text question	Minimum attendance for the end-term exam → answer with sources	
11. Follow-up	"What if I was in hospital?" using chat history; prints the search queries	Shows query rewriting
12. Image question	Answers about notice.jpg and prints the text read from it	Needs an uploaded image
13. Voice question	Transcribes voice_question.m4a and answers it	Needs an uploaded voice note
14. Directions	"Where is the Registrar's office?" – campus map added automatically	
15. Answer check	Answers all 13 test questions for manual review	3–5 min

D–E. User interfaces

Step	What the cell does	Notes
16. Chat box (ipywidgets)	Chat panel in the notebook: text, Add image, Add voice, Ask, New conversation	Works on the hub
17. Gradio page via proxy	Gradio app on a free port behind JupyterHub's proxy	Not on this hub (no jupyter-server-proxy → 404)

4. Notebook 03 – backend API and public links (21 cells, 10 code)

Step	What the cell does	Notes
1. Install	fastapi, uvicorn, python-multipart, av, requests	Once per server start
2. Settings + key	`PORT=8000`; `API_KEY` generated and saved to api_key.txt (or paste a fixed one); `PIPELINE_NOTEBOOK` = notebook 02	Key = password between page and API
3. Load pipeline	Runs 10 cells of notebook 02 found by their text (settings, corpus, context, search models, index, search, VLM, helpers, Whisper, pipeline) – no code copied	~2 min (index loaded from rag_index/)
4. The API	`GET /health`; `POST /ask` (question, session_id, image ≤10 MB, audio ≤25 MB, header X-API-Key); `POST /reset`; GPU lock = one answer at a time; per-student history (6 h)	Clean 400/401/413 errors
5. Start server	Uvicorn in a background thread on 0.0.0.0:PORT (free-port fallback) + health check	Run only this cell to restart it
6. Self-test	Asks a question, a follow-up, resets, checks a wrong key is rejected (401)	
7. Addresses	Hostname, IPs, BACKEND_URL candidates and the key	Stable: http://jupyter-<user>:8000 (Swarm service name)
8. Stop server	Does nothing unless `STOP_SERVER = True` (so Run All never stops the API)	
9. Cloudflare public link	If `MAKE_PUBLIC_LINK = True`: downloads frontend/app.py from GitHub, installs Streamlit 1.64, downloads cloudflared, starts the page next to the API, prints https://….trycloudflare.com	Works on the hub · optional `APP_PASSWORD`
10. Gradio share link	If `MAKE_GRADIO_LINK = True`: Gradio page (question, photo, microphone) with a https://….gradio.live link; shares the GPU queue	Tested locally only · optional `GRADIO_PASSWORD`

Frontend files (frontend/)

File	What it is
app.py	Streamlit chat page: history, photo attach (paperclip button), microphone recorder, clickable sources, online/busy status, new conversation, optional password screen
Dockerfile	python:3.11-slim, Streamlit 1.64 + requests, non-root user, port 8501, health check
docker-compose.yml	Portainer stack with `BACKEND_URL`, `PU_API_KEY`, `REQUEST_TIMEOUT`
.streamlit/config.toml	Navy theme, 25 MB upload limit, no usage statistics
README.md	Build → import into Portainer → deploy → connection test → troubleshooting

5. Errors and problems we hit – and the fixes

Problem	Cause	Fix
Qwen3-VL not supported	Hub had transformers 4.46	Upgrade transformers; pin torch 2.9.1 so pip keeps the CUDA build
Processor import error	Qwen3-VL's video processor needs torchvision	Install torchvision 0.24.1 (cu128) matching torch 2.9.1
pip “dependency conflict” lines	Pre-installed crewai / datasets / opentelemetry pins	Harmless – not used by the chatbot
“Restart the kernel” notice	Standard pip message	Restart after notebook 01 step 2; not needed after installing av
Everything lost after closing the server	Hub home folder is temporary	Re-upload + rerun 01 → 02 → 03 (~30–40 min); find a persistent folder
“File Changed” popup	Uploaded a new notebook while the old one was open	Click Revert (not Overwrite); close the tab before uploading
Gradio link: page not found	No jupyter-server-proxy on the hub	Chat box in the notebook; Cloudflare / Gradio share links
Port 7860 already in use	Gradio cell run twice	Pick a free port automatically; close the old server
Run All stopped the API	Last cell of notebook 03 stopped the server	Stop only when `STOP_SERVER = True`; rerun step 5 to restart
Retrieval 8/10 – “pass marks” missed	Rule split from its heading at a chunk boundary; student words ≠ regulation words (rank #127)	Previous-chunk context (#24) + query rewriting (#1)
Old 2019/2020 rules outranked 2025	Many similar regulation versions	Prompt rule: follow the newest version and name it
“Where is the Registrar?” → 6 photos	Unofficial photo captions crowded the results	`MAX_UNOFFICIAL = 3` + campus map image
58% duplicate records	Menus/footers repeated on every page	Text-hash deduplication (231,591 → 166,152)
Mirrored campus-map text	PDF stored labels reversed (RECIFFO)	Automatic un-mirroring + map image to the model
Empty teacher-transcript chunk	Segment started at 41.48 s, on a window edge	Window edges moved into pauses
People count wrong	Camera holder's finger counted; sideways footage missed people; tracker said 213 people	Ignore boxes >90% of frame height; re-count rotated blocks upright; drop the “different people” total
Files too big for GitHub	100 MB limit (V2 video 854 MB, pages 533 MB)	720p re-encode with bitrate cap (62 MB); gzip + split text files
Bad audio gave a raw decoder error	PyAV error passed through	Clean “Could not read the audio file” (400)
docker build failed	Run in D:\pu_bot (no Dockerfile)	Build inside D:\pu_bot\frontend
Hostname check failed	Run on the laptop, not the hub	Run in the JupyterHub terminal → jupyter-<user> = stable address
Portainer not found	Portainer URL is private	Ask Data Lab; meanwhile Cloudflare link

6. What we still miss – open gaps and risks

Not measured yet

- Full-corpus retrieval score with query rewriting (notebook 02 step 9) – record X/13 for the slides
- Answer speed on the H200 (tok/s from notebook 01 steps 7–8)
- Answer correctness: no graded test set yet (only 13 retrieval checks); no hallucination check
- Behaviour under load: a whole class sharing one GPU queue

Not tested on the hub

- Gradio share link (notebook 03 step 10) – tested on a laptop only
- Portainer deployment – Docker image built and tested locally; needs Portainer access

Infrastructure

- No persistent storage: the 62 GB model, index and libraries must be rebuilt after every server stop
- API key and container IP change after a restart (the jupyter-<user> name stays the same)
- One GPU, one answer at a time (~10–40 s each); no rate limiting on the public link
- Public link has no password (by choice) and passes through Cloudflare – confirm Data Lab allows it
- Chat histories live in memory only; no logs, analytics or user feedback collected

Data

- 758 scanned PDFs have no text (no OCR yet)
- Snapshot from 29 Oct 2026: fees, timetables and notices change; no scheduled re-crawl
- Media transcripts (student, teacher, videos) are automatic and unverified
- People counts = one weekday late-morning snapshot; no total of different people
- Corpus contains contact details as published on university pages (public repo)

Code and safety

- Notebook 03 finds notebook 02's cells by their text – renaming those lines breaks 03
- No automated tests or CI on the hub
- Prompt injection (instructions hidden in documents or images) not tested
- No open-source licence on the GitHub repository yet

7. Parameters you can tweak

Retrieval (notebook 02, step 1 unless noted)

Parameter	Default	What changes
`CANDIDATES`	50	FAISS hits per query sent to the reranker · ↑ finds more right chunks, slightly slower
`TOP_K`	6	Passages given to the model · ↑ for multi-part questions, ↓ for shorter, faster answers
`MAX_PER_SOURCE`	3	Chunks from one document · ↓ = more variety, ↑ = more depth from one PDF
`MAX_UNOFFICIAL`	3	Photos/videos/interviews/tour chunks allowed · ↓ = more official text
`CONTEXT_CHARS`	300	Previous-chunk text added to each chunk · changing it rebuilds the index
`EMBED_MODEL`	Qwen3-Embedding-0.6B	Bigger embedder = better recall, slower index build (rebuilds automatically)
`RERANK_MODEL`	bge-reranker-v2-m3	Cross-encoder that orders the candidates
`QUERY_PROMPT`	Instruct … Query:	Instruction for the embedder (set "" for bge-m3)
`max_seq_length` / `max_length`	512 / 512	Tokens read per passage by embedder / reranker
Passage cut-offs	2000 / 1800 chars	Text per passage for the reranker / in the model prompt (step 5, step 8)
`REBUILD_INDEX`	False	Force a rebuild after uploading a new corpus

Answer generation (notebook 02, steps 7–8)

Parameter	Default	What changes
`MODEL_ID`	Qwen3-VL-30B-A3B	The vision-language model (see section 9)
`max_new_tokens`	700 / 80 / 600	Answer length / query rewrite / image reading
`do_sample`	False (greedy)	Greedy = consistent factual answers; sampling (temperature ~0.7) = more varied wording
`SYSTEM_PROMPT`	9 rules	Tone, citation rules, “not found” behaviour, newest-regulation rule, language
`REWRITE_PROMPT`	regulation wording	How questions are rephrased for search; add more example terms
`MAX_HISTORY`	3	Past turns used for follow-ups
`load_image(max_side)`	1600 px (map 2000)	↑ reads small text better, costs more tokens/time
`NAV_WORDS` / `MAP_CLASSES`	where is, which block, …	When the campus map image is attached
`attn_implementation`	sdpa	flash_attention_2 (if installed) = faster on long prompts

Voice, API and page

Parameter	Default	What changes
`ASR_MODEL`	whisper-large-v3-turbo	Speech model (accuracy vs speed)
`generate_kwargs task`	transcribe	"translate" turns any language into English text
`PORT` / `API_KEY`	8000 / generated	API address and password (notebook 03 step 2)
`MAX_IMAGE_MB` / `MAX_AUDIO_MB`	10 / 25	Upload limits
`MAX_QUESTION_CHARS` / `SESSION_TTL_S`	2000 / 6 h	Question length; how long a student's history is kept
`REQUEST_TIMEOUT`	300 s	How long the page waits for an answer
`APP_PASSWORD` / `GRADIO_PASSWORD`	empty	Password screen for the public page

Data building (tools/)

- `MIN_CHARS = 40` – drop very short chunks · dedup by normalised text hash

- People counting: 4 frames/s, detector threshold 0.5, ignore boxes taller than 90% of the frame, 10-second windows

Tweaks to try first for better results

- Add 50–100 real student questions to `EVAL` and measure before/after every change
- If answers miss the right document: `CANDIDATES` 80–100, then try Qwen3-Embedding-4B
- Multi-part questions: `TOP_K` 8
- Keep greedy decoding for rules, fees and dates
- OCR the 758 scanned PDFs (Qwen3-VL can read page images) and rebuild the corpus

8. Tech stack

Layer	What we use	Used for
Hardware / platform	NVIDIA H200 NVL (143 GB) · Data Lab JupyterHub · Docker Swarm	Running all models; notebooks as containers
Language	Python 3.11 · Node.js 18 (crawlers)	
Deep learning	PyTorch 2.9.1 (CUDA 12.8) · torchvision 0.24.1 · Transformers (≥4.57) · Accelerate · huggingface_hub	Loading and running models
Vision-language model	Qwen3-VL-30B-A3B-Instruct (MoE, ~3B active)	Reading images, rewriting questions, answering
Embeddings	Qwen3-Embedding-0.6B via sentence-transformers	Turning text into 1024-d vectors
Vector search	FAISS `IndexFlatIP` (exact cosine)	Finding candidate chunks
Reranker	BAAI bge-reranker-v2-m3 (cross-encoder)	Ordering candidates
Speech	Whisper large-v3-turbo · PyAV (audio decoding)	Voice questions
Offline data tools	pypdf · ffmpeg · RT-DETR r50vd (people) · whisper-small + all-MiniLM-L6-v2 (pipeline)	Building the corpus
Backend	FastAPI · Uvicorn · python-multipart · requests	The API in notebook 03
User interfaces	ipywidgets · Streamlit 1.64 · Gradio	Chat box, web page, share link
Deployment	Docker (python:3.11-slim) · Portainer · Cloudflare quick tunnel (cloudflared) · Gradio share	Getting the page to students
Code hosting	Git · GitHub (public repo)	Code, data, README

9. Other models – better, cheaper or faster

GPU memory = model weights only (bf16 unless FP8); add ~5–10 GB for search models, Whisper and prompts. Quality notes are general expectations – only the default was measured in this project.

Vision-language model (MODEL_ID)

Model	GPU memory	vs default	Cost angle
Qwen3-VL-30B-A3B-Instruct (default)	62.1 GB (measured)	Baseline; fast for its size (MoE, ~3B active)	Needs an 80 GB+ GPU
Qwen3-VL-30B-A3B-Instruct-FP8	~32 GB	About the same quality	Half the memory; Hopper/Ada GPUs (H100, H200, L40S)
Qwen3-VL-32B-Instruct	~67 GB	Dense 33B; possibly stronger, slower	Same GPU class, more compute
Qwen3-VL-8B-Instruct	~18 GB	Weaker reasoning, still good at reading	Runs on 24 GB GPUs (L4, A10, RTX 4090)
Qwen3-VL-8B-Instruct-FP8	~10 GB	Close to 8B	Cheapest good option
Qwen3-VL-4B / 2B-Instruct	~9 / ~5 GB	Noticeably weaker	Tests, small GPUs
Qwen2.5-VL-7B-Instruct	~17 GB	Previous generation	Works with the same code
Qwen3-VL-235B-A22B-Instruct	~470 GB	Strongest Qwen3-VL	Multi-GPU server – not for this hub

Embeddings (EMBED_MODEL) · reranker · speech

Part	Option	Trade-off
Embeddings	Qwen3-Embedding-0.6B (default, 1024-d)	Fast: whole corpus in 7.6 min
	Qwen3-Embedding-4B (2560-d) / 8B (4096-d)	Better recall; several times slower to index; more memory
	BAAI/bge-m3 · intfloat/multilingual-e5-large-instruct	Strong multilingual alternatives (bge-m3: set `QUERY_PROMPT = ""`)
Reranker	bge-reranker-v2-m3 (default, multilingual)	Good accuracy, ~568M parameters
	bge-reranker-large · cross-encoder/ms-marco-MiniLM-L6-v2	EN/ZH · tiny (22M) and very fast, English only
Speech	whisper-large-v3-turbo (default, 809M)	Fast and accurate
	whisper-large-v3 (1.5B) · whisper-medium · whisper-small	More accurate but slower · cheaper and faster, more mistakes

Ways to cut cost

- Use FP8 models (half the memory) or 4-bit quantisation (bitsandbytes) on smaller GPUs
- Serve with vLLM: batches many students at once → far more answers per GPU-hour
- Route text-only questions to a small model (8B) and only image questions to the 30B model
- Cache answers to frequent questions (fees, attendance, calendar)
- Embedder and reranker for single queries can run on CPU; keep the GPU for the VLM
- Hosted APIs (pay per use, no GPU to run): Qwen3-VL through inference providers such as DeepInfra or Novita – data leaves the university

Recommended next upgrades

- Qwen3-VL-30B-A3B-Instruct-FP8 + vLLM → about the same quality, half the memory, much higher throughput
- Qwen3-Embedding-4B if retrieval tests still miss
- Persistent storage for the model and index → no 30–40 min setup after each restart
- Official Portainer deployment of the Streamlit page